Laboratory Guidelines Solutions

Learning Outcomes:

1. The art of experimentation
2. Experimental and analytical skills
3. Conceptual learning
4. Understanding the basis of Python and Software Defined Networks
5. Developing collaborative learning skills

Authors: Dr. Karina Gomez Chavez and Paul Zanna

Contents

Lab 1 Solution: Introduction to Python

1. Supporting Material

Possible issue on download: If you have any problem at this point with a message such as:

> An error occurred while collecting items to be installed
>
> No repository found containing:
>
> org.python.pydev/osgi.bundle/1.4.7.2843
>
> No repository found containing:
>
> org.python.pydev.ast/osgi.bundle/1.4.7.2843

that might indicate that the mirror you selected is having some network problem at that time, so, please follow the same steps with another mirror.

Installing with the zip file: The available locations for the zip files are:

- ✓ After downloading the zip file: SourceForge download
- ✓ Extract the contents of the zip file in the **eclipse/dropins** folder and restart Eclipse.

PyDev does not appear after install: The main issue at this time is that PyDev requires Java 7 in order to run. So, if you don't want to support PyDev by going the LiClipse route (which is mostly a PyDev standalone plus some goodies), you may have to go through some loops to make sure that you're actually using Java 7 to run Eclipse/PyDev (as explained below).

- ✓ Make sure you download/install the latest Java 7 JRE or JDK, try restarting to see if it got it automatically.
- ✓ I.e.: in **help > about > installation details > configuration** check if it's actually using the java 7 version you pointed at.
- ✓ If it didn't get it automatically, follow the instructions from: http://wiki.eclipse.org/Eclipse.ini to add the -vm argument to eclipse.ini on "Specifying the JVM" to specify the java 7 vm.

2. Exercise

- **Exercise 4.1.2:** Write a Hello World program
  Almost all books about programming languages start with a very simple program that prints the text Hello, World! to the screen. Make such a program in Python. Filename: hello_world.py.

```
if __name__ == '__main__':
    print("hello world")
```

- **Exercise 4.1.3:** Compute 1+1
  The first exercise concerns some very basic mathematics and programming: assign the result of 1+1 to a variable and print the value of that variable. Filename: 1plus1.py.

```
if __name__ == '__main__':
```

```
    a=1+1;
    print(a)
```

- **Exercise 4.1.4:** Type in program text

  Type the following program in your editor and execute it. If your program does not work, check that you have copied the code correctly and debug it.

```
h = 5.0 # height
r = 1.5 # radius

if __name__ == '__main__':
    area_parallelogram = h*b
    print ('The area of the parallelogram is %.3f' % area_parallelogram)
    area_square = b**2
    print ('The area of the square is %g' % area_square)
    area_circle = pi*r**2
    print ('The area of the circle is %.3f' % area_circle)
    volume_cone = 1.0/3*pi*r**2*h
    print ('The volume of the cone is %.3f' % volume_cone)
```

```
from math import pi
h = 5.0 # height
b = 2.5 # base
r = 1.5 # radius

if __name__ == '__main__':

    area_parallelogram = h*b
    print ('The area of the parallelogram is %.3f' % area_parallelogram)
    area_square = b**2
    print ('The area of the square is %g' % area_square)
    area_circle = pi*r**2
    print ('The area of the circle is %.3f' % area_circle)
    volume_cone = 1.0/3*pi*r**2*h
     print ('The volume of the cone is %.3f' % volume_cone)
```

- **Exercise 4.2.1:** Verify the use of the following operator. Execute the example code in python script and provide the output.

```
print ('Exercise 4.3.5')
print ('3 + 5 = ', 3+5)
print ("'a' + 'b'= " 'a' + 'b');
print (" -5.2 =",-5.2)
print ("50 - 24 =",50 - 24)
print ("2 * 3 =",2 * 3)
print ("'la' * 3 =",'la' * 3)
print ("3 ** 4 =",3 ** 4)
print ("13 / 3 =",13 / 3)
print ("13 // 3 =",13 // 3)
print ("-13 // 3 =",-13 // 3)
print ("13 % 3 =",13 % 3)
print ("-25.5 % 2.25 =",-25.5 % 2.25)
print ("2 << 2 =",2 << 2)
print ("11 >> 1 =",11 >> 1)
print ("5 & 3 =",5 & 3)
```

```
print ("5 | 3 =",5 | 3)
print ("5 ^ 3 =",5 ^ 3)
print ("~5 =",~5)
print ("5 < 3 is ",5 < 3)
print ("3 < 5 is ",3 < 5)
print ("5 > 3 is ",5 > 3)
x = 3; y = 6;
print ("x = 3; y = 6; x <= y is ",x <= y)
x = 4; y = 3;
print ("x = 4; y = 3 x >= 3 is ", x >= 3)
x = 2; y = 2;
print ( "x = 2; y = 2; x == y is",x == y)
x = 'str'; y = 'stR';
print ( "x = 'str'; y = 'stR'; x == y is",x == y)
x = 'str'; y = 'str';
print ("x = 'str'; y = 'str'; x == y is ",x == y)
x = 2; y = 3;
print ("x = 2; y = 3; x != y is ",x != y)
x = True;
print ( "x = True; not x is ",not x)
x = False; y = True;
print ("x = False; y = True; x and y is ", x and y)
x = True; y = False;
print ("x = True; y = False; x or y =", x or y)
```

Results:

```
Exercise 4.3.5
('3 + 5 = ', 8)
'a' + 'b'= ab
(' -5.2 =', -5.2000000000000002)
('50 - 24 =', 26)
('2 * 3 =', 6)
("'la' * 3 =", 'lalala')
('3 ** 4 =', 81)
('13 / 3 =', 4)
('13 // 3 =', 4)
('-13 // 3 =', -5)
('13 % 3 =', 1)
('-25.5 % 2.25 =', 1.5)
('2 << 2 =', 8)
('11 >> 1 =', 5)
('5 & 3 =', 1)
('5 | 3 =', 7)
('5 ^ 3 =', 6)
('~5 =', -6)
('5 < 3 is ', False)
('3 < 5 is ', True)
('5 > 3 is ', True)
('x = 3; y = 6; x <= y is ', True)
('x = 4; y = 3 x >= 3 is ', True)
('x = 2; y = 2; x == y is', True)
("x = 'str'; y = 'stR'; x == y is", False)
("x = 'str'; y = 'str'; x == y is ", True)
('x = 2; y = 3; x != y is ', True)
```

```
('x = True; not x is ', False)
('x = False; y = True; x and y is ', False)
x = True; y = False; x or y = True
```

- **Exercise 4.2.2:** The if statement:
  Create a program for taking a number from the user and check if it is the number that you have saved in the code (TIP: use input command). Save the file as if.py

```
number = 23
guess = int(input('Enter an integer : '))

if guess == number:
    # New block starts here
    print('Congratulations, you guessed it.')
    print('(but you do not win any prizes!)')
    # New block ends here
elif guess < number:
    # Another block
    print('No, it is a little higher than that')
    # You can do whatever you want in a block ...
else:
    print('No, it is a little lower than that')
    # you must have guessed > number to reach here

print('Done')
# This last statement is always executed,
                # after the if statement is executed.
```

- **Exercise 4.2.3:** The while Statement
  Create a program for taking a number from the user and check if it is the number that you have saved in the code. The program run until the user will guess the number. Save the file as ***while.py***

```
'''
Created on 1 Jul 2016

@author: e05975
'''
number = 23
running = True

while running:
    guess = int(input('Enter an integer : '))

    if guess == number:
        print('Congratulations, you guessed it.')
        # this causes the while loop to stop
        running = False
    elif guess < number:
        print('No, it is a little higher than that.')
    else:
        print('No, it is a little lower than that.')
else:
```

```
    print('The while loop is over.')
    # Do anything else you want to do here

                                print('Done')
```

- **Exercise 4.2.4:** The for Statement

  Create a program for printing a sequence of numbers. Save the file as for.py

```
'''
Created on 1 Jul 2016

@author: e05975
'''
for i in range(1, 5):
    print(i)
else:
                        print('The for loop is over')
```

3. Questions

Question 5.1: Explain what is eclipse? And why we use it for programing on python?

- Eclipse is powerful tool for programing.
- Eclipse helps in debugging the python code.

Question 5.2: Explain three main characteristics of python that you test in the lab?

- Easy to use
- Good online material
- Free and Open Source

Question 5.3: Which is the difference between empty module and main module when creating a python script?

- Empty module does not include any code, instead main has the main function code already incorporated.

Question 5.4: Find error(s) in a program

Suppose somebody has written a simple one-line program for computing sin(1):

```
x=1; print 'sin(%g)=%g' % (x, sin(x))
```

Create this program and try to run it. What is the problem?

- sin function need to be imported
- print function have incorrect format

Which is the correct code?

```
from math import sin

if __name__ == '__main__':
    #sin(1):
    x=1;
    print ('sin(%g)=%g' % (x, sin(x)))
```

Question 5.5: Create a python program that combines at least 4 operators and one statement (if, while or for)

- Test the program for each student.

Lab 2 Solution: Programing with Python

1. Exercises

- **Exercise 4.1.2:** Python function (save as function.py):

  Create python scrip using the syntax provided below.

```
def say_hello():
    # block belonging to the function
    print('hello world')
    # End of function

if __name__ == '__main__':
    say_hello()  # call the function
```

  Which is the output of this function? *hello world*

  Does the function need any parameter? *No*

- **Exercise 4.1.3:** Python function (save as function_2.py):

  Create python scrip using the syntax provided below.

```
def print_max(a, b):
    if a > b:
        print(a, 'is maximum')
    elif a == b:
        print(a, 'is equal to', b)
    else:
        print(b, 'is maximum')

if __name__ == '__main__':
    pass
    print_max(3, 4)
    # directly pass literal values
    x = 5
    y = 7
    # pass variables as arguments
    print_max(x, y)
```

  Which is the output of this function?

  4 is maximum

  7 is maximum

  Does the function need any parameter? *Yes*

- **Exercise 4.1.4:** Local variable (save as function_local.py):

  Create python scrip using the syntax provided below.

```
x = 50

def func(x):
    print('x is', x)
    x = 2
    print('Changed local x to', x)

if __name__ == '__main__':
    func(x)
    print('x is still', x)
```

  Which is the final value of variable x? *50*

Why variable x does not change to 2? Because X=2 was just defined locally in the function.

- **Exercise 4.1.5: Global variable (save as function_global.py):**

  Create python scrip using the syntax provided below.

```
x = 50

def func():
    global x
    print('x is', x)
    x = 2
    print('Changed global x to', x)

if __name__ == '__main__':
    func()
    print('Value of x is', x)
```

  Which is the final value of variable x? 2

  Why variable x change this time? Because X=2 was just defined as global in the function.

- **Exercise 4.1.6:** Python modules

  Create python scrip using the syntax provided below (save as mymodule.py).

```
def say_hi():
    print('Hi, this is mymodule speaking.')

__version__ = '0.1'
```

  Create python scrip using the syntax provided below (save as module_demo.py).

```
import mymodule

if __name__ == '__main__':
    mymodule.say_hi()
    print('Version', mymodule.__version__)
```

  Run the script, which is the role of import?

  It allows to import and use specify python modules and its functionalities in the script

  Create python scrip using the syntax provided below (save as module_demo2.py).

```
from mymodule import say_hi, __version__

if __name__ == '__main__':
    say_hi()
    print('Version', __version__)
```

  Run the script, which is the role of from, import?

  It allows to import and use specify functionalities of a module directly in the script

Section 4.2: Sockets, IPv4, and Simple Client/Server Programming

- **Exercise 4.2.1:** Printing your machine's name and IPv4 address

  Create python scrip using the syntax provided below (save as local_machine_info.py):

```
import socket

def print_machine_info():
    host_name = socket.gethostname()
    ip_address = socket.gethostbyname(host_name)
    print ("   Host name: %s" % host_name)
```

```
    print ("   IP address: %s" % ip_address)


if __name__ == '__main__':
    print_machine_info()
```

Run the script, which module the program uses? *Socket module*

Provide two additional functions of socket?

socket.getprotobyname(name)

socket.getdefaulttimeout()

- **Exercise 4.2.2:** Retrieving a remote machine's IP address

Create python scrip using the syntax provided below (save as remote_machine_info.py):

```
import socket

def get_remote_machine_info():
    remote_host = 'www.python.org'
    try:
        print ("   Remote host name: %s" % remote_host)
        print ("   IP address: %s"
%socket.gethostbyname(remote_host))
    except socket.error as err_msg:
        print ("Error accesing %s: error number and detail %s"
%(remote_host, err_msg))


if __name__ == '__main__':
    get_remote_machine_info()
```

Run the script, which is the output?

Remote host name: www.python.org

IP address: 151.101.80.223

Modify the code for getting the RMIT website info.

```
import socket

def get_remote_machine_info():
    remote_host = 'www.rmit.edu.au'
    try:
        print ("   Remote host name: %s" % remote_host)
        print ("   IP address: %s"
%socket.gethostbyname(remote_host))
    except socket.error as err_msg:
        print ("Error accesing %s: error number and detail %s"
%(remote_host, err_msg))


if __name__ == '__main__':
    get_remote_machine_info()
```

- **Exercise 4.2.3:** Converting an IPv4 address to different formats

Create python scrip using the syntax below (save as ip4_address_conversion.py):

```
import socket
from binascii import hexlify
```

```
def convert_ip4_address():
    for ip_addr in ['127.0.0.1', '192.168.0.1']:
        packed_ip_addr = socket.inet_aton(ip_addr)
        unpacked_ip_addr = socket.inet_ntoa(packed_ip_addr)
        print ("   IP Address: %s => Packed: %s, Unpacked: %s"
%(ip_addr, hexlify(packed_ip_addr), unpacked_ip_addr))

if __name__ == '__main__':
    convert_ip4_address()
```

Run the script, which is the output?

IP Address: 127.0.0.1 => Packed: b'7f000001', Unpacked: 127.0.0.1
IP Address: 192.168.0.1 => Packed: b'c0a80001', Unpacked: 192.168.0.1

How binascii works? *Convers binary values to ascii*

- **Exercise 4.2.4:** Finding a service name, given the port and protocol

Create python scrip using the syntax below (save as finding_service_name.py):

```
import socket
def find_service_name():
    protocolname = 'tcp'
    for port in [80, 25]:
        print ("Port: %s => service name: %s" %(port,
socket.getservbyport(port, protocolname)))
    print ("Port: %s => service name: %s" %(53,
socket.getservbyport(53, 'udp')))

if __name__ == '__main__':
    find_service_name()
```

Run the script, which is the output? Modify the code for getting complete the table:

```
import socket
def find_service_name():
    protocolname = 'tcp'
    for port in [21, 22, 110]:
        print ("Port: %s => service name: %s" %(port,
socket.getservbyport(port, protocolname)))
if __name__ == '__main__':
    find_service_name()
```

Port	Protocol
21	*File Transfer Protocol (FTP)*
22	*Secure Shell (SSH)*
110	*Post Office Protocol (POP3)*

- **Exercise 4.2.5:** Setting and getting the default socket timeout.

Create python scrip using the syntax below (save as socket_timeout.py):

```
import socket

def test_socket_timeout():
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    print ("Default socket timeout: %s" %s.gettimeout())
    s.settimeout(100)
    print ("Current socket timeout: %s" %s.gettimeout())
```

```
if __name__ == '__main__':
    test_socket_timeout()
```

Run the script, which is the role of socket timeout is real applications?

Socket timeouts can occur when attempting to connect to a remote server, or during communication, especially long-lived one. So socket timeout is the value that the socket wait before close it.

- **Exercise 4.**2.6: Writing a simple echo client/server application. (**Tip:** Use port 9900)

Create python scrip using the syntax below (save as echo_server.py):

```
import socket
import sys
import argparse
import codecs

from codecs import encode, decode
host = 'localhost'
data_payload = 4096
backlog = 5


def echo_server(port):
    """ A simple echo server """
    # Create a TCP socket
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    # Enable reuse address/port
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    # Bind the socket to the port
    server_address = (host, port)
    print ("Starting up echo server on %s port %s" %server_address)
    sock.bind(server_address)
    # Listen to clients, backlog argument specifies the max no. of queued
connections
    sock.listen(backlog)

    while True:
        print ("Waiting to receive message from client")
        client, address = sock.accept()
        data = client.recv(data_payload)
        if data:
            print ("Data: %s" %data)
            client.send(data)
            print ("sent %s bytes back to %s" % (data, address))
        # end connection
        client.close()


if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Socket Server Example')
    parser.add_argument('--port', action="store", dest="port", type=int,
required=True)
    given_args = parser.parse_args()
    port = given_args.port
    echo_server(port)
```

Create python scrip using the syntax below (save as echo_client.py):

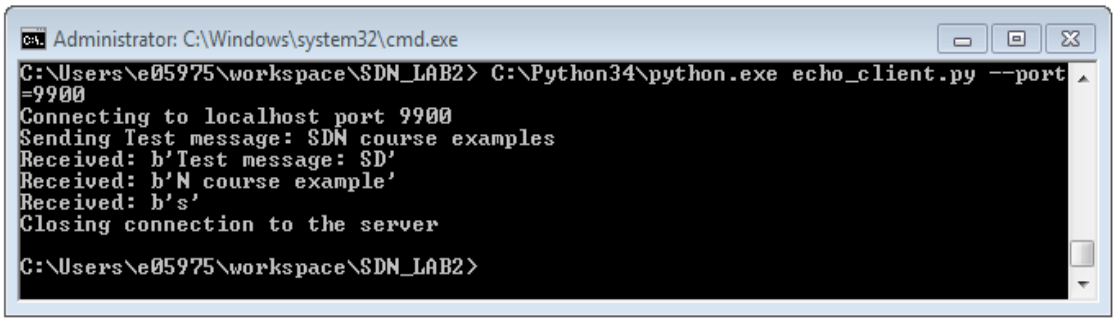
```
#!/usr/bin/env python

import socket
import sys
import argparse
import codecs

from codecs import encode, decode

host = 'localhost'


def echo_client(port):
    """ A simple echo client """
    # Create a TCP/IP socket
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    # Connect the socket to the server
    server_address = (host, port)
    print ("Connecting to %s port %s" % server_address)
    sock.connect(server_address)
    # Send data
    try:
        # Send data
        message = "Test message: SDN course examples"
        print ("Sending %s" % message)
        sock.sendall(message.encode('utf_8'))
        # Look for the response
        amount_received = 0
        amount_expected = len(message)
        while amount_received < amount_expected:
            data = sock.recv(16)
            amount_received += len(data)
            print ("Received: %s" % data)
    except socket.errno as e:
        print ("Socket error: %s" %str(e))
    except Exception as e:
        print ("Other exception: %s" %str(e))
    finally:
        print ("Closing connection to the server")
        sock.close()

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Socket Server Example')
    parser.add_argument('--port', action="store", dest="port", type=int,
required=True)
    given_args = parser.parse_args()
    port = given_args.port
    echo_client(port)
```

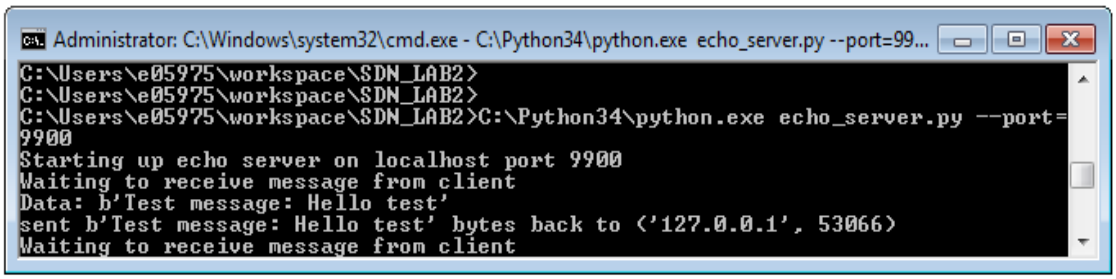
Run the script server, which is the output?

```
usage: echo_server.py [-h] --port PORT
echo_server.py: error: the following arguments are required: --port
```

What you need to do for running the program?

You need to open a terminal in the same folder of the project and run the program providing the ports





Which program you need to run first client of server?

Server, because the client connect to the server

Explain how the program works?

Client send a text "Test message: SDN course examples" to the server, server reply the same text.

What you need to do for communicate with another server in the classroom?

In the client you need to provide the host and port of the server.

2. Questions

- **Question 5.1:** Explain in your own words which are the difference between functions and modules?

  Functions are local code

  Modules are other scripts

- **Question 5.2:** Explain in your own words when to use local and global variables?

  Local inside the function, global out of the function

- **Question 5.3:** Which is the role of sockets in computing networking?

  Supports application communication

  Are the sockets defined random or there is a rule?

  Standard defined by IANA

- **Question 5.4:** Why is relevant to have the IPv4 address of remote server?

  Because it is the name of the pc in the network

  Explain what is Domain Name System (DNS)?

  The Domain Name System (DNS) is a hierarchical decentralized naming system for computers, services, or any resource connected to the Internet or a private network. It associates various information with domain names assigned to each of the participating entities. Most prominently, it translates more readily memorized domain names to the numerical IP addresses needed for the purpose of locating and identifying computer services and devices with the underlying network protocols. By providing a worldwide, distributed directory service, the Domain

Name System is an essential component of the functionality of the Internet. Mapping names with IP address.

- **Question 5.5:** Create a program that allows exchange messages increased by the user between client and server.

Server

```
#!/usr/bin/env python

import socket
import sys
import argparse
import codecs

from codecs import encode, decode
host = 'localhost'
data_payload = 4096
backlog = 5

def echo_server(port):
    """ A simple echo server """
    # Create a TCP socket
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    # Enable reuse address/port
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    # Bind the socket to the port
    server_address = (host, port)
    print ("Starting up echo server on %s port %s" %server_address)
    sock.bind(server_address)
    # Listen to clients, backlog argument specifies the max no. of queued
connections
    sock.listen(backlog)

    while True:
        print ("Waiting to receive message from client")
        client, address = sock.accept()
        data = client.recv(data_payload)
        if data:
            print ("Recived Data: %s" %data)
            message= input("Please insert you message for client: ")
            client.send(message.encode('utf_8'))
            print ("sent %s bytes back to %s" % (data, address))
        # end connection
        client.close()


if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Socket Server Example')
    parser.add_argument('--port', action="store", dest="port", type=int,
required=True)
    given_args = parser.parse_args()
    port = given_args.port
    echo_server(port)
```

Client

```
#!/usr/bin/env python

import socket
import sys
import argparse
import codecs

from codecs import encode, decode

host = 'localhost'

def echo_client(port):
    """ A simple echo client """
    # Create a TCP/IP socket
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    # Connect the socket to the server
    server_address = (host, port)
    print ("Connecting to %s port %s" % server_address)
    sock.connect(server_address)
    # Send data
    try:
        # Send data
        message = "Test message: SDN course examples"
        print ("Sending %s" % message)
        sock.sendall(message.encode('utf_8'))
        # Look for the response
        amount_received = 0
        amount_expected = len(message)
        while amount_received < amount_expected:
            data = sock.recv(16)
            amount_received += len(data)
            print ("Received: %s" % data)
    except socket.errno as e:
        print ("Socket error: %s" %str(e))
    except Exception as e:
        print ("Other exception: %s" %str(e))
    finally:
        print ("Closing connection to the server")
        sock.close()

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Socket Server Example')
    parser.add_argument('--port', action="store", dest="port", type=int,
required=True)
    given_args = parser.parse_args()
    port = given_args.port
    echo_client(port)
```

Lab 3 Solution: Python for Networking

1. Exercises

When importing a module if there is an error it means that the module needs to be installed.

- **Exercise 4.1:** Enumerating interfaces on your machine
  Create python scrip using the syntax below (save as list_network_interfaces.py):

```
#!/usr/bin/env python
import sys
import socket
import fcntl
import struct
import array

SIOCGIFCONF = 0x8912 #from C library sockios.h
STUCT_SIZE_32 = 32
STUCT_SIZE_64 = 40
PLATFORM_32_MAX_NUMBER =  2**32
DEFAULT_INTERFACES = 8

def list_interfaces():
    interfaces = []
    max_interfaces = DEFAULT_INTERFACES
    is_64bits = sys.maxsize > PLATFORM_32_MAX_NUMBER
    struct_size = STUCT_SIZE_64 if is_64bits else STUCT_SIZE_32
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    while True:
        bytes = max_interfaces * struct_size
        interface_names = array.array('B', '\0' * bytes)
        sock_info = fcntl.ioctl(sock.fileno(),SIOCGIFCONF,struct.pack('iL',
bytes, interface_names.buffer_info()[0]) )
        outbytes = struct.unpack('iL', sock_info)[0]
        if outbytes == bytes:
            max_interfaces *= 2
        else:
            break
    namestr = interface_names.tostring()
    for i in range(0, outbytes, struct_size):
        interfaces.append((namestr[i:i+16].split('\0', 1)[0]))
    return interfaces


if __name__ == '__main__':
    interfaces = list_interfaces()
          print   ("This   machine   has   %s   network   interfaces:   %s."
      %(len(interfaces), interfaces))
```

Run the script, which is the output?

This machine has 2 network interfaces: ['lo', 'eth1'].

Verify the output using the command line **ifconfig**, which is the output?

```
user@user:~ > ifconfig
eth1      Link encap:Ethernet  HWaddr c8:cb:b8:11:fa:b8
```

```
        inet6 addr: fe80::cacb:b8ff:fe11:fab8/64 Scope:Link
        UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
        RX packets:7314 errors:0 dropped:0 overruns:0 frame:0
        TX packets:7544 errors:6 dropped:0 overruns:0 carrier:6
        collisions:821 txqueuelen:1000
        RX bytes:3538382 (3.5 MB)  TX bytes:897338 (897.3 KB)
        Interrupt:20 Memory:f7f00000-f7f20000

lo      Link encap:Local Loopback
        inet addr:127.0.0.1  Mask:255.0.0.0
        inet6 addr: ::1/128 Scope:Host
        UP LOOPBACK RUNNING  MTU:65536  Metric:1
        RX packets:3562 errors:0 dropped:0 overruns:0 frame:0
        TX packets:3562 errors:0 dropped:0 overruns:0 carrier:0
        collisions:0 txqueuelen:0
        RX bytes:315171 (315.1 KB)  TX bytes:315171 (315.1 KB)
```

- **Exercise 4.2:** Finding the IP address for a specific interface on your machine
  Create python scrip using the syntax below (save as get_interface_ip_address.py):

```
import argparse
import sys
import socket
import fcntl
import struct
import array


def get_ip_address(ifname):
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    return socket.inet_ntoa(fcntl.ioctl(s.fileno(), 0x8915,
struct.pack('256s', ifname[:15]))[20:24])

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Python networking utils')
    parser.add_argument('--ifname', action="store", dest="ifname",
required=True)
    given_args = parser.parse_args()
    ifname = given_args.ifname
    print ("Interface [%s] --> IP: %s" %(ifname, get_ip_address(ifname)))
```

Run the script, which is the output?

usage: get_interface_ip_address.py [-h] --ifname IFNAME
get_interface_ip_address.py: error: argument --ifname is required

Which variable you need to provide?

user@user:~/workspace/SDN_Lab3 > python get_interface_ip_address.py --ifname eth1
Interface [eth1] --> IP: 10.0.1.135

What is the purpose of parse module?

Allow to get variable for the user.

- **Exercise 4.3:** Finding whether an interface is up on your machine
  Create python scrip using the syntax below (save as find_network_interface_status.py):

```
import argparse
import socket
import struct
import fcntl
import nmap

SAMPLE_PORTS = '21-23'

def get_interface_status(ifname):
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    ip_address = socket.inet_ntoa(fcntl.ioctl(sock.fileno(),0x8915,
struct.pack('256s', ifname[:15]))[20:24])
    nm = nmap.PortScanner()
    nm.scan(ip_address, SAMPLE_PORTS)
    return nm[ip_address].state()

if  __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Python networking utils')
    parser.add_argument('--ifname', action="store", dest="ifname",
required=True)
    given_args = parser.parse_args()
    ifname = given_args.ifname
    print ("Interface [%s] is: %s" %(ifname, get_interface_status(ifname)))
```

Run the script providing the interface to be tested, which is the output?

```
user@user:~/workspace/SDN_Lab3 > python find_network_interface_status.py --ifname
eth1
Interface [eth1] is: up
user@user:~/workspace/SDN_Lab3 > python find_network_interface_status.py --ifname
lo
Interface [lo] is: up
```

Write a script that provides the interfaces, IP and status (save as exercise43_solution.py)?

```
#!/usr/bin/env python
# Python Network Programming Cookbook -- Chapter - 3
# This program is optimized for Python 2.7.
# It may run on any other version with/without modifications.

import sys
import socket
import fcntl
import struct
import array
import argparse
import nmap


SIOCGIFCONF = 0x8912 #from C library sockios.h
STUCT_SIZE_32 = 32
STUCT_SIZE_64 = 40
PLATFORM_32_MAX_NUMBER =  2**32
DEFAULT_INTERFACES = 8

SAMPLE_PORTS = '21-23'
```

```
def list_interfaces():
    interfaces = []
    max_interfaces = DEFAULT_INTERFACES
    is_64bits = sys.maxsize > PLATFORM_32_MAX_NUMBER
    struct_size = STUCT_SIZE_64 if is_64bits else STUCT_SIZE_32
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    while True:
        bytes = max_interfaces * struct_size
        interface_names = array.array('B', '\0' * bytes)
        sock_info = fcntl.ioctl( sock.fileno(),SIOCGIFCONF, struct.pack('iL',
bytes, interface_names.buffer_info()[0]))
        outbytes = struct.unpack('iL', sock_info)[0]
        if outbytes == bytes:
            max_interfaces *= 2
        else:
            break
    namestr = interface_names.tostring()
    for i in range(0, outbytes, struct_size):
        interfaces.append((namestr[i:i+16].split('\0', 1)[0]))
    return interfaces

def get_ip_address(ifname):
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    return socket.inet_ntoa(fcntl.ioctl(s.fileno(), 0x8915,
struct.pack('256s', ifname[:15]))[20:24])


def get_interface_status(ifname):
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    ip_address = socket.inet_ntoa(fcntl.ioctl(sock.fileno(),0x8915,
struct.pack('256s', ifname[:15]))[20:24])
    nm = nmap.PortScanner()
    nm.scan(ip_address, SAMPLE_PORTS)
    return nm[ip_address].state()


if __name__ == '__main__':
    interfaces = list_interfaces()
    print "This machine has %s network interfaces: %s." %(len(interfaces),
interfaces)
    i=0
    while i<len(interfaces):
        print "Interface [%s] --> IP: %s" %(interfaces[i],
get_ip_address(interfaces[i]))
        print "Interface [%s] is: %s" %(interfaces[i],
get_interface_status(interfaces[i]))
        i=i+1
```

- **Exercise 4.4:** Detecting inactive machines on your network
  Create python scrip using the syntax below (save as detect_inactive_machines.py):

```
import argparse
import time
import sched
```

```
from scapy.layers.inet import sr, srp, IP, UDP, ICMP, TCP, ARP, Ether
#from scapy.all import sr, srp, IP, UDP, ICMP, TCP, ARP, Ether

RUN_FREQUENCY = 10

scheduler = sched.scheduler(time.time, time.sleep)


def detect_inactive_hosts(scan_hosts):
    """
    Scans the network to find scan_hosts are live or dead
    scan_hosts can be like 10.0.2.2-4 to cover range.
    See Scapy docs for specifying targets.
    """
    global scheduler
    scheduler.enter(RUN_FREQUENCY, 1, detect_inactive_hosts, (scan_hosts, ))
    inactive_hosts = []
    try:
        ans, unans = sr(IP(dst=scan_hosts)/ICMP(), retry=0, timeout=1)
        ans.summary(lambda(s,r) : r.sprintf("%IP.src% is alive"))
        for inactive in unans:
            print ("%s is inactive" %inactive.dst)
            inactive_hosts.append(inactive.dst)

        print ("Total %d hosts are inactive" %(len(inactive_hosts)))


    except KeyboardInterrupt:
        exit(0)


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description='Python networking utils')
    parser.add_argument('--scan-hosts', action="store", dest="scan_hosts",
required=True)
    given_args = parser.parse_args()
    scan_hosts = given_args.scan_hosts
    scheduler.enter(1, 1, detect_inactive_hosts, (scan_hosts, ))
    scheduler.run()
```

Add a range of IP address and run the script, which is the output? Ask your tutor for help.

This script needs python 2.7

- **Exercise 4.5:** Pinging hosts on the network with ICMP
  Create python scrip using the syntax below (save as ping_remote_host.py):

```
#!/usr/bin/env python
import os
import argparse
import socket
import struct
import select
import time
```

```
ICMP_ECHO_REQUEST = 8 # Platform specific
DEFAULT_TIMEOUT = 2
DEFAULT_COUNT = 4


class Pinger(object):
    """ Pings to a host -- the Pythonic way"""

    def __init__(self, target_host, count=DEFAULT_COUNT,
timeout=DEFAULT_TIMEOUT):
        self.target_host = target_host
        self.count = count
        self.timeout = timeout


    def do_checksum(self, source_string):
        """  Verify the packet integritity """
        sum = 0
        max_count = (len(source_string)/2)*2
        count = 0
        while count < max_count:
            val = ord(source_string[count + 1])*256 +
ord(source_string[count])
            sum = sum + val
            sum = sum & 0xffffffff
            count = count + 2

        if max_count<len(source_string):
            sum = sum + ord(source_string[len(source_string) - 1])
            sum = sum & 0xffffffff

        sum = (sum >> 16)  +  (sum & 0xffff)
        sum = sum + (sum >> 16)
        answer = ~sum
        answer = answer & 0xffff
        answer = answer >> 8 | (answer << 8 & 0xff00)
        return answer

    def receive_pong(self, sock, ID, timeout):
        """
        Receive ping from the socket.
        """
        time_remaining = timeout
        while True:
            start_time = time.time()
            readable = select.select([sock], [], [], time_remaining)
            time_spent = (time.time() - start_time)
            if readable[0] == []: # Timeout
                return

            time_received = time.time()
            recv_packet, addr = sock.recvfrom(1024)
            icmp_header = recv_packet[20:28]
            type, code, checksum, packet_ID, sequence = struct.unpack(
                "bbHHh", icmp_header
```

```
            )
            if packet_ID == ID:
                bytes_In_double = struct.calcsize("d")
                time_sent = struct.unpack("d", recv_packet[28:28 +
bytes_In_double])[0]
                return time_received - time_sent

            time_remaining = time_remaining - time_spent
            if time_remaining <= 0:
                return

    def send_ping(self, sock,  ID):
        """
        Send ping to the target host
        """
        target_addr  =  socket.gethostbyname(self.target_host)

        my_checksum = 0

        # Create a dummy heder with a 0 checksum.
        header = struct.pack("bbHHh", ICMP_ECHO_REQUEST, 0, my_checksum,
ID, 1)
        bytes_In_double = struct.calcsize("d")
        data = (192 - bytes_In_double) * "Q"
        data = struct.pack("d", time.time()) + data

        # Get the checksum on the data and the dummy header.
        my_checksum = self.do_checksum(header + data)
        header = struct.pack(
            "bbHHh", ICMP_ECHO_REQUEST, 0, socket.htons(my_checksum), ID, 1
        )
        packet = header + data
        sock.sendto(packet, (target_addr, 1))


    def ping_once(self):
        """
        Returns the delay (in seconds) or none on timeout.
        """
        icmp = socket.getprotobyname("icmp")
        try:
            sock = socket.socket(socket.AF_INET, socket.SOCK_RAW, icmp)
        except socket.error, (errno, msg):
            if errno == 1:
                # Not superuser, so operation not permitted
                msg +=  "ICMP messages can only be sent from root user
processes"
                raise socket.error(msg)
        except Exception, e:
            print "Exception: %s" %(e)

        my_ID = os.getpid() & 0xFFFF

        self.send_ping(sock, my_ID)
        delay = self.receive_pong(sock, my_ID, self.timeout)
```

```
        sock.close()
        return delay


    def ping(self):
        """
        Run the ping process
        """
        for i in xrange(self.count):
            print "Ping to %s..." % self.target_host,
            try:
                delay  =  self.ping_once()
            except socket.gaierror, e:
                print "Ping failed. (socket error: '%s')" % e[1]
                break

            if delay  ==  None:
                print "Ping failed. (timeout within %ssec.)" % self.timeout
            else:
                delay  =  delay * 1000
                print "Get pong in %0.4fms" % delay



if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Python ping')
    parser.add_argument('--target-host', action="store",
dest="target_host", required=True)
    given_args = parser.parse_args()
    target_host = given_args.target_host
    pinger = Pinger(target_host=target_host)
    pinger.ping()
```

How many function the code have?

It has 5 different functions.

Which is the command for running the script (Target host is WWW.GOOGLE.COM)?

python ping_remote_host.py --target-host www.google.com

Is there any error?

socket.error: Operation not permittedICMP messages can only be sent from root user processes

What is the solution? Which is the output?

sudo python ping_remote_host.py --target-host www.google.com

Ping to www.google.com... Ping failed. (timeout within 2sec.)

Ping to www.google.com... Ping failed. (timeout within 2sec.)

Ping to www.google.com... Ping failed. (timeout within 2sec.)

Ping to www.google.com... Ping failed. (timeout within 2sec.)

Use the script using your own IP address? Which is the output?

sudo python ping_remote_host.py --target-host 10.0.1.135

Ping to 10.0.1.135... Get pong in 0.0660ms

Ping to 10.0.1.135... Get pong in 0.0799ms

Ping to 10.0.1.135... Get pong in 0.0460ms
Ping to 10.0.1.135... Get pong in 0.0441ms

- **Exercise 4.6:** Pinging hosts on the network with ICMP using pc resources
  Create python scrip using the syntax below (save as ping_subprocess.py):

```
Created on 4 Jul 2016

@author: e05975
'''
import subprocess
import shlex

command_line = "ping -c 1 10.0.1.135"

if __name__ == '__main__':
    args = shlex.split(command_line)
    try:

subprocess.check_call(args,stdout=subprocess.PIPE,stderr=subprocess.PIPE)
        print ("Your pc is up!")
    except subprocess.CalledProcessError:
        print ("Failed to get ping.")
```

Add the IP address of your PC in the code and run the script, which is the output?

Your pc is up

Test the script with the IP of you classmate?

Ask for the classmate IP address.

What is the role of `subprocess`?

Run command lines

- **Exercise 4.7:** Scanning the broadcast of packets
  Create python scrip using the syntax below (save as broadcast_scanning.py):

```
from scapy import all
from scapy.layers.inet import sr, srp, IP, UDP, ICMP, TCP, ARP, Ether,
sniff
import os
captured_data = dict()

END_PORT = 1000

def monitor_packet(pkt):
    if IP in pkt:
        if not captured_data.has_key(pkt[IP].src):
            captured_data[pkt[IP].src] = []

    if TCP in pkt:
        if pkt[TCP].sport <=  END_PORT:
            if not str(pkt[TCP].sport) in captured_data[pkt[IP].src]:
                captured_data[pkt[IP].src].append(str(pkt[TCP].sport))

    os.system('clear')
    ip_list = sorted(captured_data.keys())
    for key in ip_list:
```

```
        ports=', '.join(captured_data[key])
        if len (captured_data[key]) == 0:
            print ('%s' % key)
        else:
            print ('%s (%s)' % (key, ports))

if __name__ == '__main__':
    sniff(prn=monitor_packet, store=0)
```

Run the script, which is the output?

IP Address from the machines that are sending broadcast packets.

- **Exercise 4.8:** Sniffing packets on your network
  Tcpdump is a common packet analyzer that runs under the command line. It allows the user to display TCP/IP and other packets being transmitted or received over a network to which the computer is attached. Distributed under the BSD license,[3] tcpdump is free software.
  - *Open a linux terminal and check the usage of tcpdump using the command line tcpdum –help*

    [-c count]
    [-C file_size] [-G rotate_seconds] [-F file]
    [-i interface] [-j tstamp_type] [-m module] [-M secret]
    [--number] [-Q in|out|inout]
    [-r file] [-V file] [-s snaplen] [-T type] [-w file]
    [-W filecount]
    [-E spi@ipaddr algo:secret,...]
    [-y datalinktype] [-z postrotate-command] [-Z user]
    [--time-stamp-precision=tstamp_precision]
    [--immediate-mode] [--version]
    [expression]
  - Using tcpdump get the traffic present in the Ethernet interface of your pc (10 packet only), which is the command line?

    'tcpdump -i eth1 -c 1 -v'
  - Using the `subprocess` write a program for sniffing 1 packet of the Ethernet interface? (save as `packet_sniffer.py`).

```
import subprocess
import shlex
import time

tcpdumpProcess = subprocess.Popen([ 'tcpdump', '-i', 'eth1', '-c','1', '-
v'], stdout=subprocess.PIPE,stderr=subprocess.PIPE)
time.sleep(3)
tcpdumpProcess.terminate()
#tcpdump_stdout = tcpdumpProcess.communicate()[0]
stdout,stderr = tcpdumpProcess.communicate()
#print tcpdump_stdout
print (stdout,stderr)
```

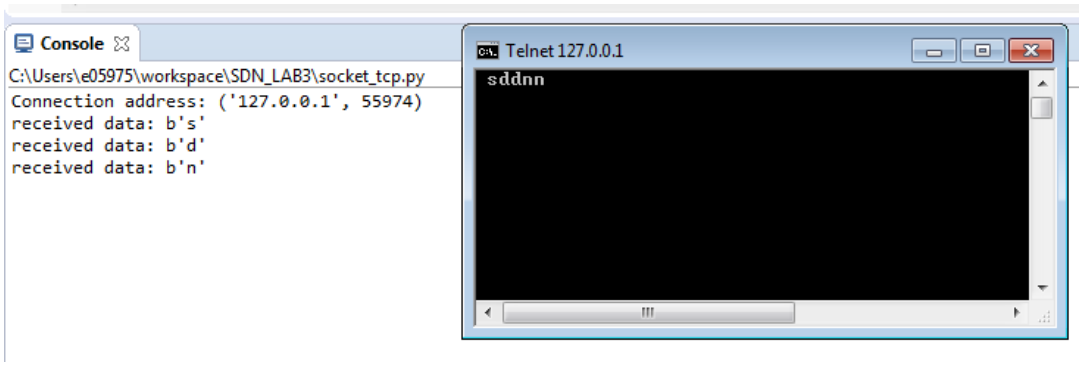
- **Exercise 4.9:** Performing a basic Telnet
  Create python scrip using the syntax below (save as echo_client.py):

```
#!/usr/bin/env python

import socket

TCP_IP = '127.0.0.1'
TCP_PORT = 62
BUFFER_SIZE = 20  # Normally 1024, but we want fast response

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind((TCP_IP, TCP_PORT))
s.listen(1)

conn, addr = s.accept()
print ('Connection address:', addr)
while 1:
    data = conn.recv(BUFFER_SIZE)
    if not data: break
    print ("received data:", data)
    conn.send(data)  # echo
conn.close()
```

Add the IP address of your PC in the code and run the script, which is the output?

Console is waiting for connection

- Open a Linux terminal and execute "telnet 'IP PC' port", what happen with the server console?



- Test the script with the IP of you classmate?

  Ask the classmate for IP address and test

2. Questions

- **Question 5.1:** Explain in your own words what is a network interface?

  Physical connection to network interface
- **Question 5.2:** How many network interface usually you find in your pc?
  1. *Wireless*
  2. *Ethernet*
  3. *Loopback*
- **Question 5.3:** Explain why you sniffing the network interface? Give examples?

In order to capture the packets circulating in the network, for example for debugging a protocol.

- **Question 5.4:** Explain why it is relevant to communicate using sockets?

Provide an organized way of communicate.

- **Question 5.5:** Provide a script that allows you to get information from internet? For example reading new? Reading Wikipedia? Reading Google map?
  Test the scripts of the students.

Lab 4 Solution: SDN Controllers and Mininet

1. Exercises

Section 4.1: Using traffic Generators

- **Exercise 4.1.1:** Open a Linux terminal, and execute the command line *iperf --help*. Provide four configuration options of iperf.

  Client/Server:
  -f, --format [kmKM] format to report: Kbits, Mbits, KBytes, MBytes
  -i, --interval seconds between periodic bandwidth reports
  -l, --len [KM] length of buffer to read or write (default 8 KB)
  -m, --print_mss print TCP maximum segment size (MTU - TCP/IP header)

- **Exercise 4.1.2:** Open two Linux terminals, and configure terminal-1 as client (*iperf –c IPv4_server_address*) and terminal-2 as server (*iperf -s*). **Note: use the loopback address.** Which are the statistics provided at the end of transmission?

  The time interval, the amount of data transfer and the bandwidth.

- **Exercise 4.1.3:** Open two Linux terminals, and configure terminal-1 as client and terminal-2 as server for exchanging UDP traffic, which are the command lines?

  iperf -c 127.0.0.1 -u
  iperf -s -u

Which are the statistics are provided at the end of transmission?

The time interval, the amount of data transfer, the bandwidth, the jitter and the datagram loss percentage.

What is different from the statistics provided in exercise 4.1.1.

The jitter and the datagram loss percentage, because TCP guaranties 0% packet loss.

- **Exercise 4.1.4:** Open two Linux terminals, and configure terminal-1 as client and terminal-2 as server for exchanging UDP traffic, with:
  - Packet length = 1000bytes
  - Time = 20 seconds
  - Bandwidth = 1Mbps
  - Port = 9900

  Which are the command lines?

iperf -c 127.0.0.1 -u -l 1000 -t 20 -b 1 -p 9900

iperf -s -u -p 9900

Exercise 4.1.5: Open Wireshark (work with your partner for this task),

- Navigator -> Internet->Wireshark
- Capture all the packets in the Ethernet interface (Capture->ethx->apply)
- Student 1: Open one Linux terminal, and configure terminal-1 as server for exchanging UDP traffic for 10s in port 9900.
- Student 2: Open one Linux terminal, and configure terminal-1 as client (use IPv4 address of Student 1) for exchanging UDP traffic for 10s in port 9900 (use a packet length of 1280 bytes).
- Check the capture packets in Wireshark, how you can visualize the traffic exchanged with student 1?

Filtering the packets by the destination address.

- **Exercise 4.1.5:** Using iperf find the saturation point of the Ethernet segment (work with your partner for this task). Ask your tutor for help.

Students need to exchange UDP packets until produce at least 50% of packet lost.

Section 4.2: Using Mininet (Note that if you run the mininet command without specifying a controller, it will use the ovsc controller, by default)

- **Exercise 4.2.1:** Open two Linux terminals, and execute the command line *ifconfig* in terminal-1. How many interfaces are present?

Two interfaces: Loopback and Ethernet.

In terminal-2, execute the command line *sudo mn*, which is the output?

**** Creating network*
**** Adding controller*
**** Adding hosts:*
h1 h2
**** Adding switches:*
s1
**** Adding links:*
(h1, s1) (h2, s1)
**** Configuring hosts*

h1 h2
**** Starting controller*
c0
**** Starting 1 switches*
s1 ...
**** Starting CLI:*
mininet>

In terminal-1 execute the command line *ifconfig*. How many real and virtual interfaces are present now?

Two interfaces: Loopback and Ethernet.
Three virtual interfaces: Ethernet.

- **Exercise 4.2.2:** Interacting with mininet; in terminal-2, display the following command lines and explain what it does:
  - mininet> help

  Help for using mininet
  - mininet> nodes

  Description of the nodes
  - mininet> net

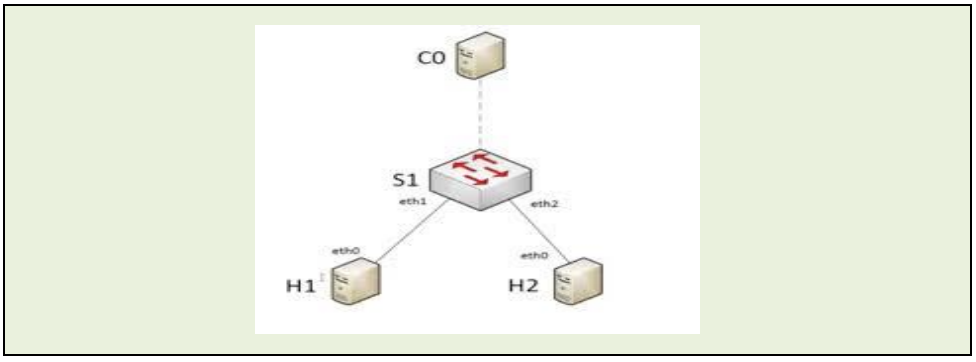
  Description of the network
  - mininet> dump

  Details of the network nodes IP address.
  - mininet> h1 ifconfig –a

  Configuration of host 1
  - mininet> s1 ifconfig –a

  Configuration of host 2
  - mininet> h1 ping -c 5 h2

  ping between h1 and h2
  - Draw the network topology that is created in mininet:



In terminal-2, display the following command line: sudo ovs-vsctl show, what is displayed?

48305d97-3805-4861-9f9e-311da3eb7ef3
Bridge "s1"
Controller "ptcp:6634"
Controller "tcp:127.0.0.1:6633"
is_connected: true
fail_mode: secure
Port "s1"
Interface "s1"
type: internal
Port "s1-eth2"
Interface "s1-eth2"

Port "s1-eth1"
Interface "s1-eth1"
ovs_version: "2.0.2"
The controller configuration.

- **Exercise 4.2.3:** In terminal-2, display the following command line: *sudo mn --link tc,bw=10,delay=500ms*
  - mininet> h1 ping -c 5 h2, What happen with the link?
    PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
    64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=4007 ms
    64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=3001 ms
    64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=2000 ms
    64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=2000 ms
    64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=2000 ms
    The delay of the link increase.
  - mininet> h1 iperf –s –u &
  - mininet> h2 iperf –c IPv4_h1 -u , Is there any packet loss?
    No

  Modify iperf for creating packet loss in the mininet network, which is the command line?
    h2 iperf -c 10.0.0.1 -u -b 11M –l 1000
    11% of packet loss

Section 4.3: Using and configuring your controller

- **Exercise 4.3.1:** Local controller: Mininet can be used with python scripts as well. The following script reproduce same architecture as before: (save as .py)
  In terminal-1, run the script. The script will open four terminals; run the following command line in the terminals:
  1. terminal-2 C0 (*ifconfig*), get the virtual interface and run (tcpdump -i s1-ethx arp)
  2. terminal-3 S1 (*ifconfig*), get the virtual interface and run (tcpdump -i s1-ethx arp)
  3. terminal-4 H1 (*ifconfig*), get the virtual interface and run (tcpdump -i h1-ethx arp)
  4. terminal-5 H2 (*ifconfig*), get the virtual interface and run (tcpdump -i h2-ethx arp)

  In terminal-1, display the following command lines:
  - mininet> nodes
  - mininet> net
  - mininet> dump
  - mininet> h1 ping -c 5 h2

  Which is the role of controller with ARP packets?
    Controller will provide the mapping of IP address and MAC address, then it will install the tables in the S1 for indicated in which port the h1 and h2 are connected

  In terminal-1, display the following command line:
  - mininet> h1 ping -c 5 h2

  What happen this time? Why?
    ARP messages are not sending again because the S1 know already where the hosts are connected.
- **Exercise 4.3.1:** Remote controller: The following script reproduce same architecture as before using an external controller: (save as .py)

This scrip defines the behavior of the controller: (save as simple_switch_13.py, it is also available in blackboard)

In terminal-1, run the script. The script open four Linux terminals, run the following command line in the terminals:

1. terminal-2 C0 (*ifconfig*), get the virtual interface and run (tcpdump -i s1-ethx arp)
2. terminal-3 S1 (*ifconfig*), get the virtual interface and run (tcpdump -i s1-ethx arp)
3. terminal-4 H1 (*ifconfig*), get the virtual interface and run (tcpdump -i h1-ethx arp)
4. terminal-5 H2 (*ifconfig*), get the virtual interface and run (tcpdump -i h2-ethx arp)

In terminal-1, display the following command lines:

- mininet> h1 ping -c 5 h2

What happen? Why?

Ping in not reachable becouse Controller is not running

Open another terminal-5 run *sudo ovs-vsctl set Bridge s1 protocols=OpenFlow13*

In terminal terminal-2 Controller run `ryu-manager ./simple_switch_13.py`

In terminal-1, display the following command lines:

- mininet> h1 ping -c 5 h2

What happen? Why?

Controller is now running so Controller will provide the mapping of IP address and MAC address, then it will install the tables in the S1 for indicated in which port the h1 and h2 are connected

2. Questions

Question 5.1: Explain how the traffic generators work?

Use the base concept of server and client for exchange systemic traffic.

Question 5.2: Provide another example of traffic generator and how it work.

Any traffic generator like JPT or other online

Question 5.3: Which is the main difference between configuring UDP and TCP traffic?

UDP traffic if fully customized in terms of packet lengh and batwidth. Instead TCP only allow customize the amount of traffic.

Question 5.4: In your opinions which are the main advantages and disadvantages of working with mininet? Provide at least 3.

Advantage:

1. *free*
2. *easy to install*
3. *allow test scalability*

Disadvantage:

1. *It is not real devices*
2. *No real traffic can be injected*
3. *No INTERNET output.*

Question 5.5: Explain the architecture of Software defined Networks?

- Controller role: *Brain of the network*
- Switch role: *Hands of the network*
- Host role: *Clients*

Question 5.6: What is the advantage of having a programmable Controller?

Fully customize, program and controll your network.

Question 5.7: Search for another Controller and provide the description and features.

Any other controller like POX or HPE.

Question 5.6: Provide the python code for emulation the following network.

Test the code provided for students.

Lab 5 Solution: Zodiac OpenFlow Switch (Configure)

1. Exercises

Exercise 4.1.4: Using the *Base Functionalities* command lines in Z-CLI, check the information about each Ethernet port. Which is the status of the port? Which ports are OpenFlow? Are all the ports in the same VLAN?

Zodiac_FX# show ports

Port 1
Status: DOWN
VLAN type: OpenFlow
VLAN ID: 100

Port 2
Status: DOWN
VLAN type: OpenFlow
VLAN ID: 100

Port 3
Status: DOWN
VLAN type: OpenFlow
VLAN ID: 100

Port 4
Status: DOWN
VLAN type: Native
VLAN ID: 200

Exercise 4.1.5: Using the *OpeFLow Functionalities* command lines in Z-CLI, check the information about flows? Is there any flow? If not, why?

Zodiac_FX(openflow)# show flows
No Flows installed!
There is no controller, therefore no flows are there.

Exercise 4.1.6: Using the *Config Functionalities* command lines in Z-CLI, check the Zodiac FX configuration, complete the following information:

Zodiac_FX(config)# show config

Configuration
Name: Zodiac_FX
MAC Address: 70:B3:D5:6C:D1:0B
IP Address: 10.0.1.99
Netmask: 255.255.255.0
Gateway: 10.0.1.1

OpenFlow Controller: 10.0.1.8
OpenFlow Port: 6633
Openflow Status: Enabled
Failstate: Secure
Force OpenFlow version: Disabled
Stacking Select: MASTER
Stacking Status: Unavailable

Is the MAC address equal to the one provided in the hardware? Yes

Display the status of the VLANS, in which VLAN the controller should be connected?

Zodiac_FX(config)# show vlans

VLAN ID	*Name*	*Type*
100	*'Openflow'*	*OpenFlow*
200	*'Controller'*	*Native*

Exercise 4.1.6: Connect the Ethernet cable between the Zodiac FX native port and the lab-PC as show in the following figure.

Show the status of the ports again using Z-CLI (same as **Exercise 4.1.4**), is the status of port 4 changed?

Zodiac_FX# show ports

Port 1
Status: DOWN
VLAN type: OpenFlow
VLAN ID: 100

Port 2
Status: DOWN
VLAN type: OpenFlow
VLAN ID: 100

Port 3
Status: DOWN
VLAN type: OpenFlow
VLAN ID: 100

Port 4
Status: UP
VLAN type: Native
VLAN ID: 200

In the lab PC open a terminal and ping the zodiac device (ping -c 5 10.0.1.99), does the ping works? No.

If not, why? *The Controller Ip need to be configured.*

Exercise 4.1.7: Configuring lab PC to communicate with Zodiac FX. In order to establish the communication between Zodiac FX and Lab-PC using native port, Lab-PC has to be configured as Controller. Therefore, configure lab-PC with the following static IP address:

1. *IP Address: 10.0.1.8*
2. *Netmask: 255.255.255.0*
3. *Gateway: 10.0.1.1*

In the lab PC open a terminal, ping the zodiac device again (ping -c 5 10.0.1.99), does the ping works? Yes
If yes, why? We configured the controller.

Section 4.2: Changing Zodiac configuration.

Exercise 4.2.1 Using the *Config Functionalities* command lines in Z-CLI, set Zodiac IP address to 10.0.1.(10+x), where x is the number of your Zodiac FX (check the box). Save the configuration in the non-voltaic memory.

```
Zodiac_FX(config)# set ip-address 10.0.1.8.11
IP Address set to 10.0.1.11
Zodiac_FX(config)# show config
------------------------------------------------------------------------
Configuration
 Name: Zodiac_FX
 MAC Address: 70:B3:D5:6C:D1:0B
 IP Address: 10.0.1.11
 Netmask: 255.255.255.0
 Gateway: 10.0.1.1
 OpenFlow Controller: 10.0.1.8
 OpenFlow Port: 6633
 Openflow Status: Enabled
 Failstate: Secure
 Force OpenFlow version: Disabled
 Stacking Select: MASTER
 Stacking Status: Unavailable
------------------------------------------------------------------------

Zodiac_FX(config)# save
Writing Configuration to EEPROM (200 bytes)
Done!
```

Exercise 4.2.2: In the lab PC open a terminal, ping the zodiac device again using the new IP address, does the ping works? *Yes.*

Exercise 4.2.1: Using the *Config Functionalities* command lines in Z-CLI, create a new vlan the name SDN_lab and type openflow (Vlan-ID=300). Then add the port 3 to the new vlan. Save the configuration in the non-voltaic memory.

```
Zodiac_FX(config)# add vlan 300 SDN_lab
Added VLAN 300 'SDN_Lab'
```

Zodiac_FX(config)# set vlan-type 300 openflow
VLAN 300 set as OpenFlow
Zodiac_FX(config)# delete vlan-port 100 3
Port 3 has been removed from VLAN 100
Zodiac_FX(config)# add vlan-port 300 3
Port 3 is now assigned to VLAN 300

When done display *show vlans* and *show ports* and ask your tutor for marking it.

Zodiac_FX(config)# show vlans

VLAN ID	*Name*	*Type*
100	*'Openflow'*	*OpenFlow*
200	*'Controller'*	*Native*
300	*'SDN_Lab'*	*OpenFlow*

Zodiac_FX# show vlan ports

Port 1
Status: DOWN
VLAN type: OpenFlow
VLAN ID: 100

Port 2
Status: DOWN
VLAN type: OpenFlow
VLAN ID: 100

Port 3
Status: DOWN
VLAN type: OpenFlow
VLAN ID: 300

Port 4
Status: UP
VLAN type: Native
VLAN ID: 200

---

2. Questions

Question 5.1: Explain the difference between the Native and OpenFlow ports?

Native port is a normal IP instead OpenFlow port will works under OpenFlow protocol

Question 5.2: Why we cannot use dynamic IP between zodiac and lab-PC?

Because zodiac and controller IP address are configured in the code as static address.

Question 5.3: What is the difference between and OpenFlow and non-OpenFow Switch (apart from the support of OpenFlow)

The architecture of the network is different; an OpenFlow switch works only when a controller is active.

Question 5.4: Provided others examples of commercial open flow switch.

HP, Huawei any open-flow switch is valid.

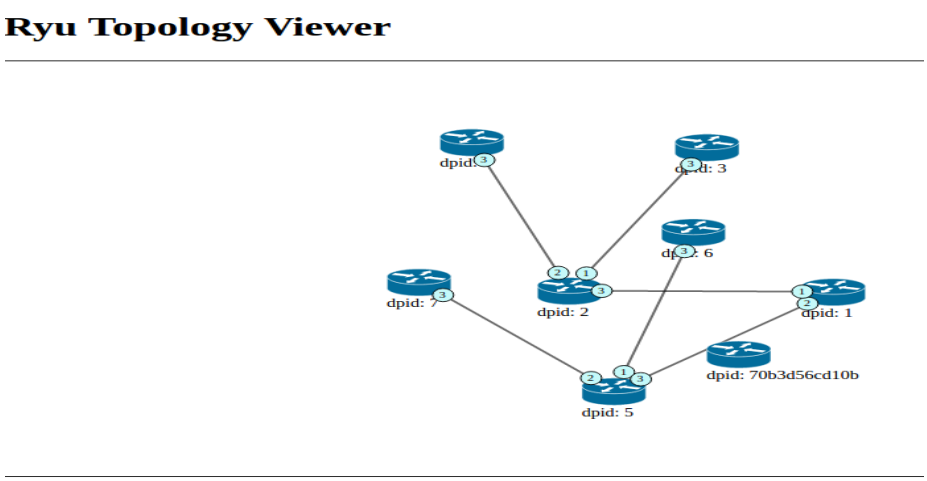
Lab 6 Solution: OpenFlow Protocol

1. Exercises

Section 4.1: Using Graphical interface of the RYU controller. The ryu.app.gui_topology provides topology visualization.

Exercise 4.1.3: Which is the topology displayed by the RYU GUI? Are the hosts displayed in the RYU GUI?

Tree topology, host and controller are not displayed in the topology.



Exercise 4.1.4: Using the script, create other two simple topologies?

Any topology produced by the students is acceptable.

Section 4.2: Real network

Exercise 4.2.2: Is Zodiac in the topology displayed by the RYU GUI?

Yes it is,

Is zodiac connected with the virtual network?

Not, virtual and real are separated.

Exercise 4.2.3: (Work in group of three students for this section) Create a tree topology using the Zodiac FX switches. Is the topology display in the GUI?

No

If not, why?

PC controller has only one Ethernet port, if the student manages is possible to use the two Ethernet interfaces,

Which port should use for interconnecting the switch? Explain.

The OpenFlow ports, native ports are for direct connection with the Controller.

Exercise 4.2.4: Create at least two different topologies using with Zodiac FX and show to your tutor, What you need to configure?

No configuration is needed, it is just use the OpenFlow ports.

.

Section 4.3: OpenFlow Protocol (Work individual)

Exercise 4.3.1: Connect to the Z-CLI, is there any flow?
No flows are in Zodiac FX because there is no controller

Exercise 4.3.2: Run RYU manager (NOTE: this is another way to run the scrip provided in lab 4)*: ryu-manager --verbose ryu.app.simple_switch_13*
Is there any flow now?
One flow is in Zodiac FX
What is the meaning of this flow?
Direct the traffic to the controller
Which is the action?
Controller

Exercise 4.3.3: From Student A PC ping Student B PC, is there any new flow?
Two new flows are in Zodiac FX
What is the role of the new flow?
Connect the port-1 and port-2
Which is the action?
Send to port-1 and port-2

Exercise 4.3.4: Check the status of zodiac port, is there any statistic?
In Z-CLI: show ports
There is several statistics per port about transmit and receive packets

Exercise 4.3.5: Clear the flows Zodiac-FX and perform the experiment again, this time capture the message exchanged between zodiac and controller and identify the packet-in and packet-out messages.
Using wireshark or tcpdump
What is the format of the messages?
TCP format
Which fields the messages has?
The typical TCP open flow messages, students need to list the fields

Exercise 4.3.6: Switch the Ethernet cables of port 1 with port 2 and check the flow, is there any change?
The flow disappears after a timeout, otherwise student need to clean the flows.
Ping the Student A PC from Student B PC what happen with the flows?
New flows are created

Exercise 4.3.7: Switch the cable of port 1 to port 3 and check the flow, is there any change?
The flow disappears after a timeout, otherwise student need to clean the flows.
Ping the Student A PC from Student B PC, is the ping working?
Yes
What happen with the flows?
New flows will be istalled.

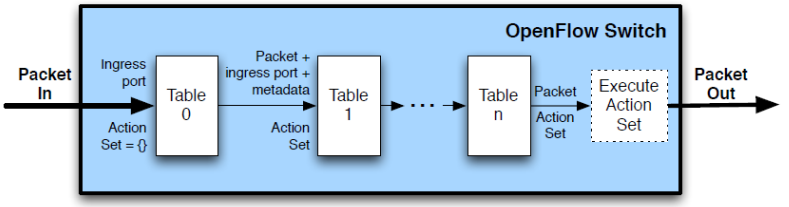
2. Questions

Question 5.1: How the RYU GUI interface can be improved? Provide some ideas.
Provide an interface to add or remove flow
Provide the configuration of the switches, ports, IP address Vlans.

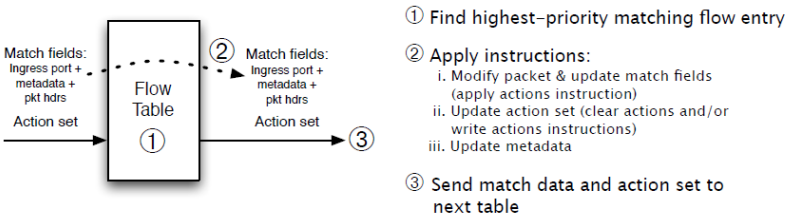
Question 5.2: Explain at least two the advantage and disadvantage of using mininet or Zodiac FX?

	Mininet	Zodiac
Advantage	Scalability and flexibility in terms of topology	Real hardware provide a better learning experience
Disadvantage	Difficult to visualize conflicts	Limited number or physical ports

Question 5.3: Explain how the open flow tables are created?



(a) Packets are matched against multiple tables in the pipeline



(b) Per-table packet processing

Figure 2: Packet flow through the processing pipeline

Question 5.4: When Zodiac FX receive a packet, which functions an OpenFlow Switch performs:

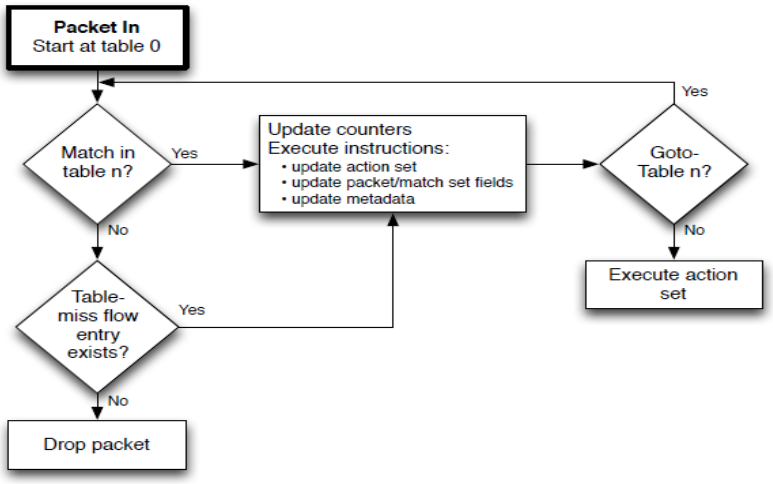


Figure 3: Flowchart detailing packet flow through an OpenFlow switch.

Question 5.6: Provide your personal opinion about OpenFlow protocol? Focus on the functionalities and flexibility.

Let's see how students comment on the flexibility of OpenFlow

Lab 7 Solution: Controller Rest API

3. Exercises

Section 4.1: Using Rest API of RYU controller, perform the following exercises.

Exercise 4.1.2: Run ryu controller including the two applications i) simple switch and 2) RYU.APP.OFCTL_REST using the following command line:

sudo ryu-manager ryu.app.simple_switch_13 ryu.app.ofctl_rest

Using ZODIAC_CLI check if there is any flow?

Yes, the default flow sending all the packets to the controller.

Exercise 4.1.3: Get the list of all switches which connected to the controller using the following command line:

sudo curl -X GET http://0.0.0.0:8080/stats/switches

The output is the ZODIAC_ dpid_number, provide the output.

Each Zodiac has a different number

Exercise 4.1.4: Get all flows stats of the switch which specified with Datapath ID in URI using the following command line:

sudo curl -X GET http:// 0.0.0.0:8080/stats/flow/ ZODIAC_ dpid_number

Provide and explain the output.

Output is the statistics about the flows

Exercise 4.1.5: Get ports stats of the switch which specified with Datapath ID in URI using the following command line:

sudo curl -X GET http:// 0.0.0.0:8080/stats/port/ ZODIAC_ dpid_number

Provide and explain the output.

Output is the statistics about the ports

Section 4.2: Adding and removing flows

Exercise 4.2.1: Add a flow entry to the switch using the following command line:

```
curl -X POST -d '{
   "dpid": ZODIAC_ dpid_number,
   "cookie": 1,
   "cookie_mask": 1,
   "table_id": 0,
   "idle_timeout": 30,
   "hard_timeout": 30,
   "priority": 11111,
   "flags": 1,
   "match":{
      "in_port":1
   },
```

```
    "actions":[
       {
          "type":"OUTPUT",
          "port": 2
       }
    ]
 }' http://0.0.0.0:8080/stats/flowentry/add
```

Using ZODIAC-CLI check if there is any additional flow?

It is the new flow

What is the meaning of the flow?

Link the port 1 to port 2

Exercise 4.2.2: Create a command line for adding a flow entry that allows to link port 2 with port 1. Which is the command line?

```
curl -X POST -d '{
   "dpid": ZODIAC_ dpid_number,
   "cookie": 1,
   "cookie_mask": 1,
   "table_id": 0,
   "idle_timeout": 30,
   "hard_timeout": 30,
   "priority": 11111,
   "flags": 1,
   "match":{
      "in_port":2
   },
   "actions":[
      {
         "type":"OUTPUT",
         "port": 1
      }
   ]
}' http://0.0.0.0:8080/stats/flowentry/add
```

Provide the screenshot of the ZODIAC flow table.

Three flows should be visualized

Exercise 4.2.3: Delete all flow entries of the switch which specified with Datapath ID in URI using the following command line:

curl -X DELETE http://0.0.0.0:8080/stats/flowentry/clear/ ZODIAC_ dpid_number

Using ZODIAC-CLI what happen with the flows?

All the flows are deleted

Exercise 4.2.4: Add a group entry to the switch using the following command line:

```
curl -X POST -d '{
   "dpid": ZODIAC_ dpid_number,
   "type": "ALL",
   "group_id": 1,
   "buckets": [
     {
        "actions": [
          {
             "type": "OUTPUT",
             "port": 1
          }
        ]
     }
   ]
 }' http://0.0.0.0:8080/stats/groupentry/add
```

Using ZODIAC-CLI check the new flow?

Three is a new flow

Which is the meaning of the flow?

All the packets go to port 1

4. Questions

Question 5.1: Explain the advantages of REST API of the Controller.

Flexible

Easy to use

Faster

Question 5.2: Explain and test two commands lines (different from the ones used during the lab) of RYU.APP.OFCTL_REST.

Usage and description of command lines are available at http://ryu.readthedocs.io/en/latest/app/ofctl_rest.html is valid

Question 5.3: Provide two real service examples for explaining the usage of reactive flow instantiation.

Any service that require authentication or security check is valid

Question 5.4: Provide two real service examples for explaining the usage of proactive flow instantiation.

Any service that does not authentication or security check is valid

Any service with short delay requirement is valid

Question 5.5: what is the difference between SDN and OpenFlow?

OpenFlow is just a protocol of SDN.